Conception et évaluation d’un SOC augmenté par l’intelligence artificielle

Intégration de Wazuh et de Qwen3 pour l’analyse des événements de sécurité

Rapport technique

Ahmed Chegueni

SOC | Wazuh | Qwen3 | Ollama

Table des matières

Introduction générale **1**

1 Conception et déploiement d'un SOC augmenté par l'IA **2**
1.1 Architecture du laboratoire 2
1.2 Déploiement et collecte des événements avec Wazuh 3
1.3 Intégration de Qwen3 et de l'application Agentic AI 5
1.3.1 Installation et exécution locale d'Ollama 5
1.3.2 Démarrage de l'application Agentic AI 6

2 Expérimentations et interprétation des alertes par l'IA **8**
2.1 Méthodologie et critères d'évaluation 8
2.2 Échecs d'authentification Windows 9
2.3 Force brute SSH rapide : analyse d'une attaque déjà reconnue 9
2.3.1 Détection d'une attaque SSH par force brute 9
2.3.2 Interprétation, investigation et recommandations de l'agent IA . . 10
2.4 SSH low-and-slow : reconnaissance d'un comportement non corrélé 12
2.4.1 Principe et déroulement du test 12
2.4.2 Événements observés dans Wazuh 13
2.4.3 Analyse obtenue avec Qwen3 4B 14
2.4.4 Modification d'une tâche cron et surveillance FIM 14
2.5 Bilan des expérimentations et perspectives d'amélioration 16

Conclusion générale **19**

Table des figures

1.1 Architecture du laboratoire SOC augmenté par l'intelligence artificielle . 4
1.2 Interface de déploiement d'un agent Wazuh Linux pour Debian-CIBLE . 4
1.3 Paramètres de connexion et de fonctionnement de l'application Agentic AI 7

2.1 Exemple d'événement Windows 4625 collecté par Wazuh 9
2.2 Détection de l'attaque SSH par force brute dans Wazuh 10
2.3 Investigation d'une attaque SSH par Qwen3 et historique des appels aux outils Wazuh 11
2.4 Analyse d'une attaque SSH confirmée et recommandations techniques proposées par Qwen3 12
2.5 Simulation de dix tentatives SSH low-and-slow depuis Kali Linux 13
2.6 Alertes individuelles d'échec d'authentification SSH enregistrées par Wazuh sur Debian-CIBLE 13
2.7 Verdict de Qwen3 4B identifiant une possible force brute SSH malgré l'absence d'alerte corrélée observée 14
2.8 Détection de la création et de la modification d'un fichier par le module FIM de Wazuh 15
2.9 Investigation de l'alerte FIM par Qwen3 et historique des appels aux outils 16

Introduction générale

La sécurité des systèmes d'information constitue aujourd'hui un enjeu majeur pour les organisations, confrontées à des menaces informatiques de plus en plus nombreuses et complexes. La supervision des infrastructures repose notamment sur la collecte et l'analyse des événements de sécurité. Dans ce contexte, les systèmes de gestion des informations et des événements de sécurité (SIEM), tels que Wazuh, permettent de centraliser les journaux et de générer des alertes à partir de règles de détection.

Cependant, ces solutions présentent certaines limites, notamment lorsqu'il s'agit d'identifier des comportements suspects nécessitant une mise en relation de plusieurs événements. L'analyse et l'interprétation des alertes restent ainsi des tâches essentielles pour les analystes des centres opérationnels de sécurité (SOC).

L'intelligence artificielle constitue une approche complémentaire permettant d'assister les analystes dans leurs investigations. Les modèles de langage peuvent notamment faciliter l'interprétation des journaux, identifier des comportements suspects et proposer une analyse contextualisée des événements. Leur utilisation soulève néanmoins des questions concernant la fiabilité des résultats et les risques d'erreurs d'interprétation.

C'est dans ce contexte que s'inscrit ce projet, dont l'objectif est de concevoir un environnement SOC associant le SIEM open source Wazuh à un agent d'intelligence artificielle reposant sur le modèle Qwen3, exécuté localement avec Ollama. La problématique retenue est la suivante : dans quelle mesure l'intégration d'un agent d'intelligence artificielle à Wazuh peut-elle enrichir l'analyse des événements de sécurité et améliorer l'identification de comportements suspects ?

Pour répondre à cette problématique, un environnement virtualisé a été mis en place afin de simuler différentes activités de sécurité et d'étudier leur interprétation par l'agent IA à partir des alertes collectées par Wazuh.

Ce rapport présente d'abord l'architecture et le déploiement de l'environnement SOC augmenté par l'IA, puis les expérimentations réalisées et les analyses obtenues. Enfin, il examine les apports, les limites et les perspectives d'amélioration de cette intégration.

Chapitre 1

Conception et déploiement d'un SOC augmenté par l'IA

Introduction

Ce chapitre présente l'environnement technique mis en place pour produire, centraliser et interpréter des événements de sécurité. L'objectif n'est pas seulement de déployer Wazuh, mais de construire une chaîne complète reliant des systèmes supervisés à un agent IA capable d'interroger les alertes et d'en proposer une analyse.

1.1 Architecture du laboratoire

Le laboratoire a été mis en place à l'aide de VMware, qui permet d'exécuter plusieurs machines virtuelles sur un même ordinateur physique. Ces machines sont interconnectées à travers un réseau virtuel privé utilisant la plage d'adresses `192.168.157.0/24`. Cette architecture permet de reproduire un environnement informatique supervisé et de réaliser des expérimentations de sécurité dans un cadre contrôlé.

L'environnement comprend principalement un serveur de supervision Wazuh, une machine Kali Linux destinée à générer les activités de test, une machine Debian jouant le rôle de cible et un poste Windows sur lequel fonctionne l'intelligence artificielle.

Le serveur Wazuh, installé sur Debian à l'adresse `192.168.157.131`, constitue l'élément central de la supervision. Il regroupe les composants Wazuh Manager, Indexer et Dashboard, qui assurent respectivement la réception et l'analyse des événements, leur stockage et leur consultation à travers une interface web.

La machine Kali Linux, dont l'adresse est `192.168.157.132`, est utilisée pour générer des activités de sécurité contrôlées. Ces actions sont principalement dirigées vers la machine `Debian-CIBLE`, située à l'adresse `192.168.157.134`. Cette dernière héberge notamment un service SSH ainsi qu'un agent Wazuh identifié par le numéro `002`. L'agent collecte les événements du système et les transmet au serveur central .

Le poste Windows héberge l'environnement d'intelligence artificielle composé d'Ollama, du modèle de langage Qwen3 et d'une application Python développée avec Flask. Cette application permet à l'agent IA d'interroger Wazuh et de présenter les résultats de ses investigations à travers une interface web.

Table 1.1 – Composants du laboratoire

Composant	Adresse IP	Rôle principal
Serveur Wazuh	192.168.157.131	Centralisation, analyse, stockage et visualisation des événements de sécurité.
Kali Linux	192.168.157.132	Génération d'activités de test à destination de la machine cible.
Debian-CIBLE	192.168.157.134	Machine supervisée hébergeant SSH et l'agent Wazuh 002.
Windows-SOC-Lab	192.168.157.1	hébergement local de l'agent IA avec Ollama et Flask.

Le fonctionnement de cette architecture repose sur deux étapes complémentaires. Dans un premier temps, les agents Wazuh installés sur les machines supervisées transmettent les événements collectés au serveur central. Wazuh traite ces informations à l'aide de ses règles de détection et génère des alertes lorsqu'un comportement correspond aux conditions définies.

Dans un second temps, l'application Agentic AI, exécutée sur Windows, communique avec les interfaces de Wazuh afin de récupérer les alertes pertinentes. Ces informations sont ensuite transmises au modèle Qwen3, exécuté localement grâce à Ollama. Le modèle peut alors interpréter les événements, rechercher des relations entre plusieurs alertes et proposer des hypothèses sur les comportements observés.

Cette organisation permet d'associer les mécanismes de détection de Wazuh aux capacités d'interprétation de l'intelligence artificielle. Il convient toutefois de préciser que l'agent IA ne surveille pas directement le trafic réseau : ses analyses dépendent des événements collectés par Wazuh et des informations accessibles à travers les outils d'investigation.

La figure 1.1 illustre les différents composants du laboratoire ainsi que leurs principales interactions.

1.2 Déploiement et collecte des événements avec Wazuh

Le déploiement utilise Wazuh 4.14.8 dans une architecture regroupant Manager, Indexer et Dashboard. Le Manager reçoit les événements et applique les règles de dé-

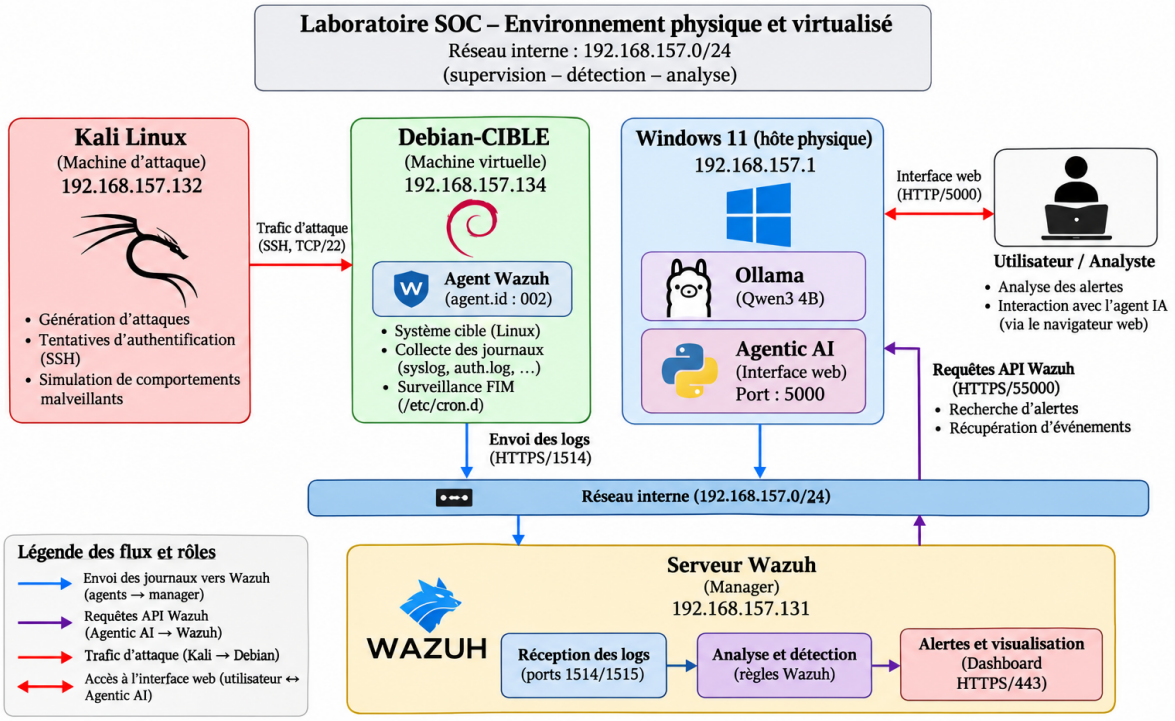


Figure 1.1 – Architecture du laboratoire SOC augmenté par l'intelligence artificielle

Figure 1.1 – Architecture du laboratoire SOC augmenté par l'intelligence artificielle

tection ; l'Indexer conserve les documents ; le Dashboard permet leur consultation. La machine Debian cible a été enregistrée comme agent 002, puis la remontée des événements a été vérifiée depuis l'interface.

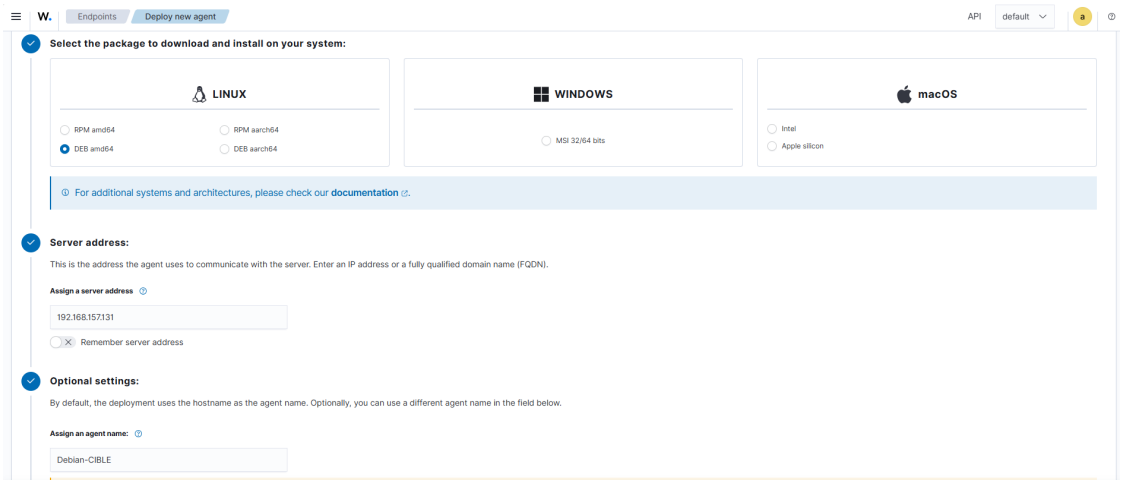


Figure 1.2 – Interface de déploiement d'un agent Wazuh Linux pour Debian-CIBLE

La supervision de la machine Debian-CIBLE repose principalement sur la collecte des journaux système et la surveillance de l'intégrité des fichiers. Les journaux d'authentification SSH permettent à Wazuh d'identifier les tentatives de connexion échouées et de générer des alertes à partir des événements enregistrés par le système.

En parallèle, le module File Integrity Monitoring (FIM) permet de surveiller les fichiers et répertoires sensibles afin de détecter d'éventuelles créations, modifications ou suppressions.

Ces mécanismes permettent à Wazuh de centraliser des événements de sécurité horodatés et structurés, qui servent ensuite de base aux investigations de l'agent d'intelligence artificielle. L'objectif est d'évaluer la capacité de Qwen3 à interpréter ces informations, à établir des liens entre plusieurs événements et à identifier des comportements suspects.

1.3 Intégration de Qwen3 et de l'application Agentic AI

L'intelligence artificielle est exécutée localement à l'aide d'Ollama. Les modèles Qwen3 1.7B et Qwen3 4B ont été utilisés durant les essais. Une application Python/Flask, issue d'un prototype Agentic AI, expose une interface web permettant de soumettre une question d'investigation. L'agent dispose d'outils de recherche et d'agrégation des alertes Wazuh. Le modèle choisit des appels d'outils, exploite les réponses obtenues et rédige un verdict.

L'application Agentic AI propose une interface de type chatbot permettant à l'utilisateur d'interroger les alertes de sécurité collectées par Wazuh en langage naturel. Selon la demande, l'agent IA recherche les événements en fonction de plusieurs paramètres, notamment l'identifiant de la machine supervisée, la période d'analyse, le niveau de gravité des alertes et les règles de détection associées. Les informations récupérées sont ensuite transmises au modèle Qwen3, qui les interprète afin d'identifier des comportements suspects, d'établir des liens entre les événements et de proposer des pistes d'investigation.

L'application dispose également d'un mode d'investigation automatique, configurable pour analyser périodiquement les nouvelles alertes sans intervention manuelle. Cette fonctionnalité permet d'envisager une supervision régulière assistée par l'intelligence artificielle, en complément des mécanismes de détection de Wazuh.

1.3.1 Installation et exécution locale d'Ollama

Ollama est installé sur le poste Windows hôte à l'aide du gestionnaire de paquets `winget`. Dans un terminal PowerShell, la commande suivante permet d'installer l'application :

```
winget install --id Ollama.Ollama -e
```

Après l'installation, le modèle retenu pour les analyses principales est téléchargé localement :

```
ollama pull qwen3:4b
```

Le modèle plus léger, utilisé lors des premiers essais, peut également être récupéré :

```
ollama pull qwen3:1.7b
```

Pour les expérimentations, l'exécution a été forcée sur le processeur afin d'éviter les problèmes rencontrés avec l'accélération graphique du poste. Les commandes ci-dessous sont lancées dans une session PowerShell, en conservant le serveur Ollama actif pendant l'analyse :

```
Get-Process ollama* -ErrorAction SilentlyContinue |
    Stop-Process -Force
$env:OLLAMA_LLM_LIBRARY="cpu"
ollama serve
```

L'exécution du modèle est vérifiée dans un second terminal à l'aide de la commande `ollama ps`. Cette commande permet de consulter les modèles actuellement chargés en mémoire ainsi que les ressources matérielles utilisées pour leur exécution.

Dans notre configuration, l'indication `100% CPU` signifie que le modèle Qwen3 fonctionne entièrement sur le processeur, sans utiliser de carte graphique pour les calculs. Cette configuration permet d'exécuter l'intelligence artificielle localement, avec des performances qui dépendent principalement de la puissance du processeur et de la mémoire disponible.

1.3.2 Démarrage de l'application Agentic AI

L'application Flask utilise un fichier de configuration `.env` qui regroupe les paramètres nécessaires à son fonctionnement et à la communication avec les différents composants du laboratoire.

Comme le montre la figure 1.3, le paramètre `OLLAMA_MODEL` permet de sélectionner le modèle d'intelligence artificielle utilisé, ici `qwen3:4b`, tandis que `OLLAMA_HOST` définit l'adresse du service Ollama exécuté localement.

Les paramètres `WAZUH_HOST` et `INDEXER_HOST` indiquent respectivement les adresses de l'API Wazuh et de l'Indexer, permettant à l'agent IA de récupérer les alertes et les événements de sécurité. Les identifiants associés assurent l'authentification auprès de ces services.

Enfin, les paramètres `UI_HOST` et `UI_PORT` définissent l'adresse et le port de l'interface web Flask.



Figure 1.3 – Paramètres de connexion et de fonctionnement de l'application Agentic AI

Une fois ces paramètres configurés, l'application Flask est lancée dans un autre terminal PowerShell depuis le répertoire du projet, à l'aide des commandes suivantes :

```
cd C:\Users\AdminX\Documents\SOC-AI\Agentic-AI
$env:AGENTIC_MAX_STEPS="3"
C:\Users\AdminX\Documents\SOC-AI\.venv\Scripts\python.exe .\app.py
```

L'interface web devient alors accessible localement à l'adresse `http://127.0.0.1:5000`. Le paramètre `AGENTIC_MAX_STEPS`, fixé à 3, limite le nombre d'étapes d'investigation que l'agent IA peut effectuer lors d'une requête. Cette limitation permet de maîtriser le temps de traitement, notamment lorsque le modèle Qwen3 fonctionne sur le processeur.

Ces commandes correspondent à l'environnement de test utilisé, dans lequel l'application et son environnement virtuel Python ont été préalablement installés.

Conclusion

Le laboratoire permet de relier des événements de sécurité Linux à un agent IA local. La suite du rapport examine cette intégration en conditions expérimentales, avec une attention particulière portée à l'interprétation des alertes plutôt qu'à leur seule apparition dans Wazuh.

L'utilisation du mécanisme de *tool calling* permet à l'agent IA d'interroger Wazuh et d'exploiter les événements collectés pour conduire ses investigations. Les expérimentations permettront d'évaluer sa capacité à identifier des comportements suspects et à proposer des recommandations techniques pertinentes.

Chapitre 2

Expérimentations et interprétation des alertes par l'IA

Introduction

Ce chapitre constitue le cœur du projet. Chaque scénario est analysé selon la même démarche : génération contrôlée de l'activité, vérification des événements Wazuh, interrogation de Qwen3 et confrontation de son verdict aux preuves disponibles. Cette méthode permet de distinguer l'intérêt d'une interprétation contextualisée des simples capacités de détection par règles.

2.1 Méthodologie et critères d'évaluation

Les tests ont été réalisés dans le laboratoire isolé décrit au chapitre précédent. L'agent IA reçoit une consigne de recherche ciblant une machine supervisée et, lorsque cela est possible, une période précise. Les réponses sont examinées selon quatre critères : l'identification du comportement, l'exactitude des faits (adresses IP, utilisateurs, nombre de tentatives et chronologie), la pertinence des hypothèses de sécurité et la fiabilité des conclusions.

L'objectif est d'évaluer dans quelle mesure l'intelligence artificielle peut enrichir les analyses de Wazuh. En effet, certaines attaques peuvent générer plusieurs alertes individuelles sans être explicitement identifiées comme une attaque globale par les règles de détection du SIEM. Dans ces situations, l'agent IA peut aider l'analyste à reconnaître un comportement suspect en établissant des liens entre différents événements et en reconstituant leur chronologie. Il peut ainsi contribuer à mettre en évidence des attaques qui n'ont pas fait l'objet d'une alerte spécifique dans Wazuh.

Cette approche repose sur la complémentarité entre Wazuh et Qwen3 : le SIEM collecte et détecte les événements, tandis que l'IA apporte une couche supplémentaire d'interprétation et d'investigation. Les conclusions proposées par le modèle sont ensuite

évaluées à partir des informations réellement présentes dans les alertes, afin de distinguer les observations vérifiables des hypothèses formulées par l'intelligence artificielle.

2.2 Échecs d'authentification Windows

Une première série d'essais a porté sur l'authentification d'un compte local `SOC-Test`. Les tentatives infructueuses sont visibles dans les journaux Windows et remontent dans Wazuh. Le document de preuve présente notamment l'événement Windows `4625`, associé à un échec d'ouverture de session et au compte ciblé. Dans l'extrait conservé, la règle Wazuh `60122` est de niveau 5.



Figure 2.1 – Exemple d'événement Windows 4625 collecté par Wazuh

Ce test confirme que les données nécessaires à l'étude des échecs d'authentification peuvent être centralisées. En revanche, la capture ne suffit pas à démontrer qu'une attaque par force brute Windows a été reconnue, ni à évaluer une réponse de Qwen3 pour ce scénario. Il sert donc de validation de la collecte, sans lui attribuer un résultat IA non documenté.

2.3 Force brute SSH rapide : analyse d'une attaque déjà reconnue

2.3.1 Détection d'une attaque SSH par force brute

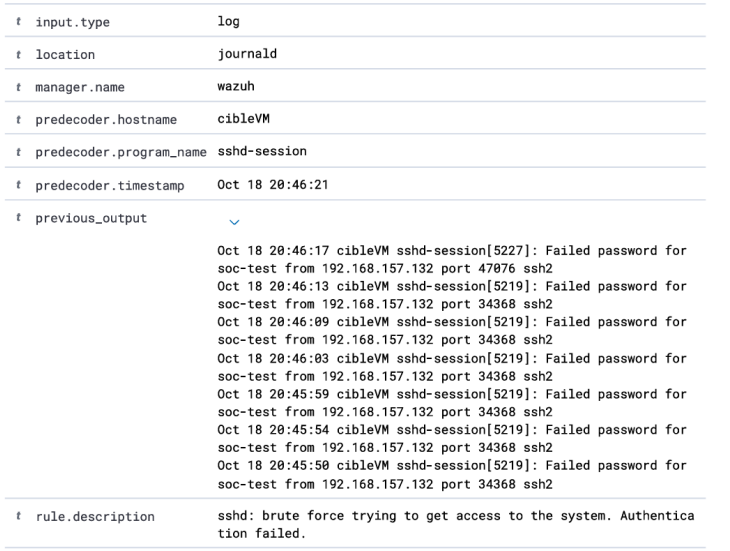
La machine Kali Linux (`192.168.157.132`) a été utilisée pour simuler une attaque par force brute contre le service SSH de la machine `Debian-CIBLE` (`192.168.157.134`).

L'outil Hydra permet d'automatiser les tentatives d'authentification en testant plusieurs mots de passe sur un même compte utilisateur. Dans notre environnement de laboratoire, la commande utilisée pour reproduire ce scénario est de la forme suivante :

```
hydra -l soc-test -P passwords.txt \
  ssh://192.168.157.134
```

Cette commande cible le poste Debian avec une liste de mots de passe contenue dans le fichier `passwords.txt`. Les tentatives successives génèrent des événements d'échec d'authentification dans les journaux SSH de Debian.

La concentration de ces événements a déclenché la règle Wazuh `5763`, de niveau 10, associée à la technique MITRE ATT&CK `T1110` (Brute Force). L'alerte indique notamment l'adresse IP source `192.168.157.132` et le compte utilisateur ciblé.



t input.type	log
t location	journald
t manager.name	wazuh
t predecoder.hostname	cibleVM
t predecoder.program_name	sshd-session
t predecoder.timestamp	Oct 18 20:46:21
t previous_output	Oct 18 20:46:17 cibleVM sshd-session[5227]: Failed password for soc-test from 192.168.157.132 port 47076 ssh2 Oct 18 20:46:13 cibleVM sshd-session[5219]: Failed password for soc-test from 192.168.157.132 port 34368 ssh2 Oct 18 20:46:09 cibleVM sshd-session[5219]: Failed password for soc-test from 192.168.157.132 port 34368 ssh2 Oct 18 20:46:03 cibleVM sshd-session[5219]: Failed password for soc-test from 192.168.157.132 port 34368 ssh2 Oct 18 20:45:59 cibleVM sshd-session[5219]: Failed password for soc-test from 192.168.157.132 port 34368 ssh2 Oct 18 20:45:54 cibleVM sshd-session[5219]: Failed password for soc-test from 192.168.157.132 port 34368 ssh2 Oct 18 20:45:50 cibleVM sshd-session[5219]: Failed password for soc-test from 192.168.157.132 port 34368 ssh2
t rule.description	sshd: brute force trying to get access to the system. Authentication failed.

Figure 2.2 – Détection de l'attaque SSH par force brute dans Wazuh

2.3.2 Interprétation, investigation et recommandations de l'agent IA

Deux tests ont été réalisés pour évaluer l'apport de Qwen3 dans l'analyse d'une attaque SSH par force brute.

Dans le premier test, l'agent IA a été invité à examiner les alertes de `Debian-CIBLE` sans information préalable sur l'attaque. Il a identifié les échecs d'authentification répétés et conclu à une tentative de force brute.

Cette investigation repose sur le mécanisme de *tool calling*, qui permet au modèle d'interroger Wazuh à travers des fonctions externes. Comme le montre la figure 2.3, Qwen3 a appelé `search_alerts` pour récupérer les alertes de l'agent 002, puis `get_agent_timeline`

pour examiner leur chronologie. Les données obtenues ont servi à produire son interprétation.

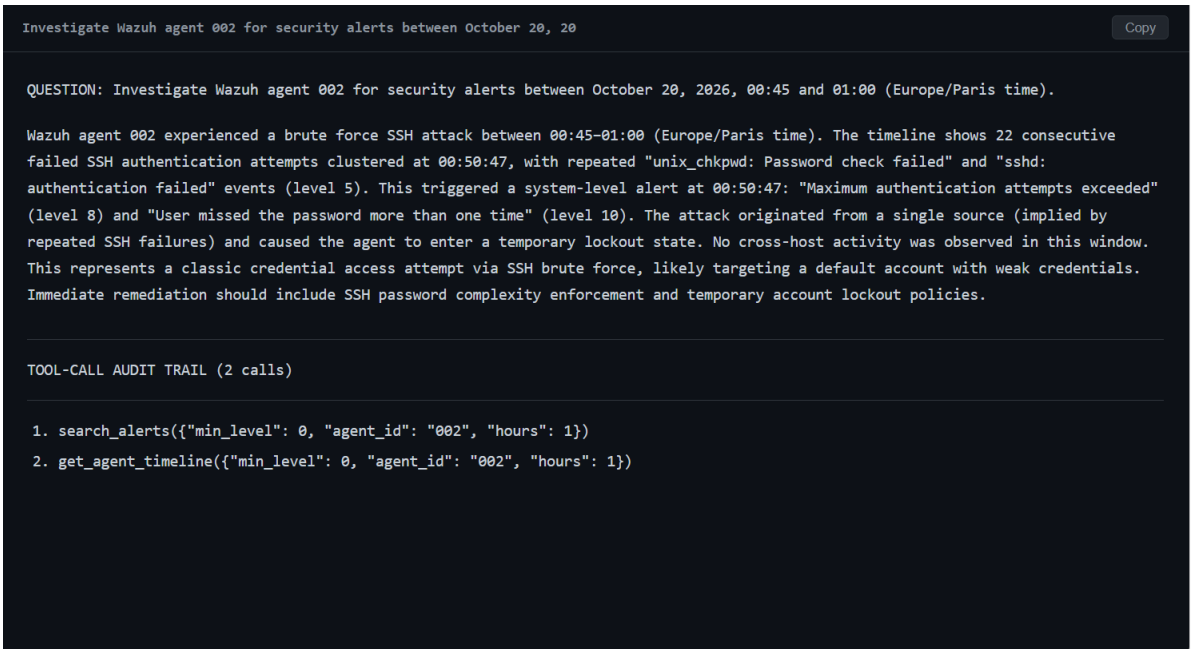


Figure 2.3 – Investigation d'une attaque SSH par Qwen3 et historique des appels aux outils Wazuh

Cette approche peut faciliter le travail d'un analyste SOC supervisant plusieurs hôtes, en synthétisant les alertes et en mettant en évidence les comportements suspects.

Dans le second test, l'attaque était déjà identifiée. Les informations fournies à Qwen3 comprenaient la règle Wazuh `5763`, le niveau 10, la technique MITRE ATT&CK `T1110`, l'adresse IP de Kali Linux et le compte `soc-test`. L'objectif était d'évaluer son assistance à la réponse à incident.

Comme le montre la figure 2.4, Qwen3 a généré un rapport structuré décrivant l'attaque, les machines concernées et les mesures de protection recommandées : blocage de l'adresse IP source, authentification SSH par clés et surveillance des nouvelles tentatives de connexion.

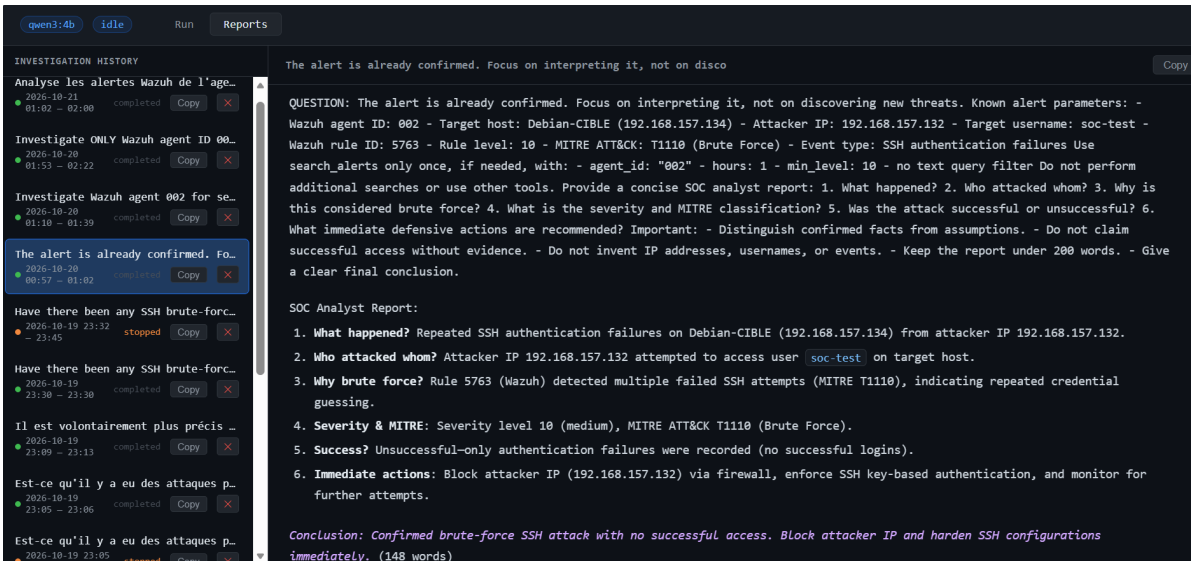


Figure 2.4 – Analyse d'une attaque SSH confirmée et recommandations techniques proposées par Qwen3

Ces expérimentations mettent en évidence deux usages complémentaires de l'agent IA : l'investigation des événements de sécurité et l'assistance à la réponse à incident. Les recommandations proposées restent soumises à la validation de l'analyste SOC.

2.4 SSH low-and-slow : reconnaissance d'un comportement non corrélé

2.4.1 Principe et déroulement du test

Une force brute classique concentre de nombreuses tentatives dans un intervalle court. À l'inverse, une activité dite *low-and-slow* espace les essais afin de rester sous certains seuils de détection temporels. Pour étudier cette situation, dix connexions SSH avec des mots de passe incorrects ont été tentées depuis Kali vers `Debian-CIBLE`, avec une pause programmée de 40 secondes entre les essais, comme l'illustre la figure 2.5.

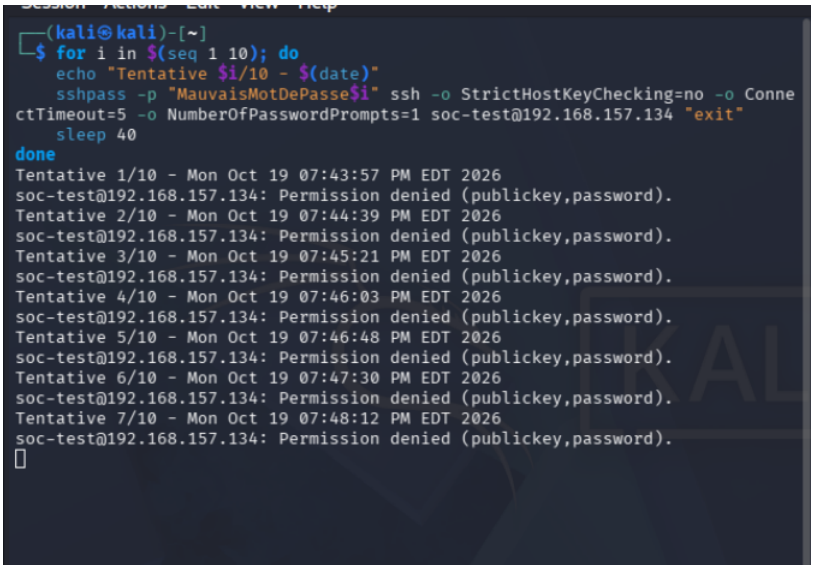


Figure 2.5 – Simulation de dix tentatives SSH low-and-slow depuis Kali Linux

2.4.2 Événements observés dans Wazuh

Lors des tentatives d'authentification SSH sur `Debian-CIBLE`, Wazuh a enregistré plusieurs alertes de niveau 5, comme le montre la figure 2.6.

Chaque alerte possède un identifiant de règle (*Rule ID*) et un niveau de gravité (*Rule Level*). Les règles `5557`, `5503` et `5760` correspondent respectivement à un échec de vérification du mot de passe, un échec d'authentification PAM et un échec d'authentification SSH. Le niveau 5 indique un événement de sécurité nécessitant une attention, sans constituer à lui seul une preuve d'attaque.

	Oct 19, 2026 @ 22:55:10.327	Debian-CIBLE	sshd: authentication failed.	5	5760
	Oct 19, 2026 @ 22:55:08.337	Debian-CIBLE	PAM: User login failed.	5	5503
	Oct 19, 2026 @ 22:55:08.335	Debian-CIBLE	unix_chkpwd: Password check failed.	5	5557

Figure 2.6 – Alertes individuelles d'échec d'authentification SSH enregistrées par Wazuh sur Debian-CIBLE

Ces trois alertes, enregistrées à quelques secondes d'intervalle, illustrent les différentes étapes de traitement d'un échec d'authentification.

Dans ce scénario, aucune alerte de force brute associée à la règle `5763` n'a été observée. Wazuh a correctement enregistré les échecs, mais ne les a pas regroupés en une alerte signalant une attaque globale. Cette observation met en évidence une limite de corrélation des événements dans les conditions du test, et non un défaut de collecte des journaux.

2.4.3 Analyse obtenue avec Qwen3 4B

L'agent IA a été interrogé sur les événements de l'agent `002` dans une fenêtre correspondant au test. La réponse de Qwen3 4B a identifié une tentative de force brute SSH, mentionné dix échecs d'authentification et relevé l'adresse source `192.168.157.132`. Le modèle a également indiqué qu'aucune connexion réussie n'était établie par les événements consultés.

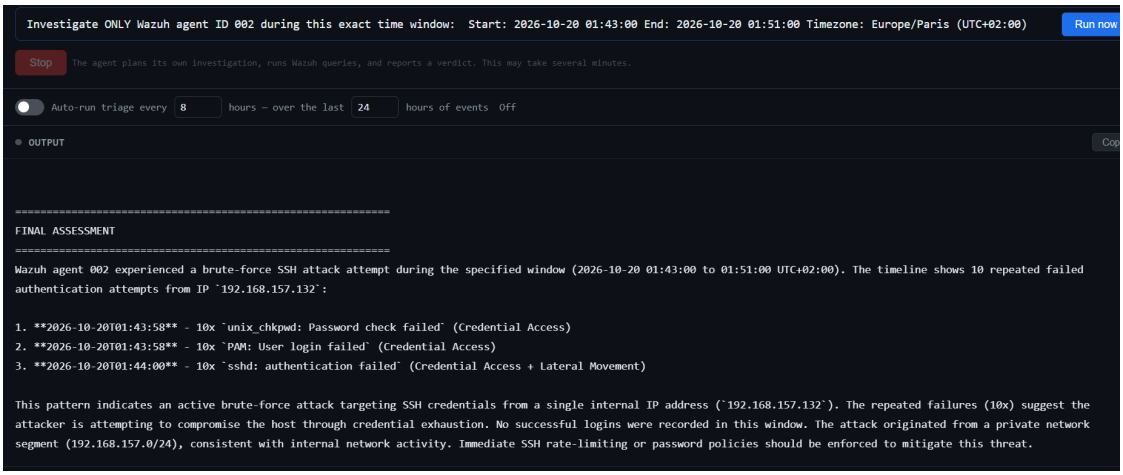


Figure 2.7 – Verdict de Qwen3 4B identifiant une possible force brute SSH malgré l'absence d'alerte corrélée observée

Ce résultat constitue l'un des apports les plus intéressants du projet. Alors que Wazuh a enregistré les échecs d'authentification individuellement sans générer d'alerte spécifique de force brute, l'agent IA a réussi à identifier un comportement suspect compatible avec une attaque SSH à faible fréquence, caractérisée par des tentatives espacées dans le temps.

Cette capacité montre que l'intelligence artificielle peut contribuer à réduire les faux négatifs, en mettant en évidence des attaques qui ne sont pas explicitement reconnues par les règles de corrélation du SIEM. L'IA apporte ainsi une couche d'analyse complémentaire, capable de rapprocher plusieurs événements et d'alerter l'analyste sur une menace potentiellement passée inaperçue.

2.4.4 Modification d'une tâche cron et surveillance FIM

Ce dernier scénario consiste à simuler une technique de persistance sur la machine `Debian-CIBLE`. Une tâche cron a été créée dans le répertoire `/etc/cron.d`, puis modifiée pour exécuter périodiquement un script de test situé dans `/tmp`. Ce script réalise uniquement des opérations de journalisation locale.

Pour détecter ces changements, le module *File Integrity Monitoring* (FIM) de Wazuh a été configuré pour surveiller le répertoire `/etc/cron.d` en temps réel.

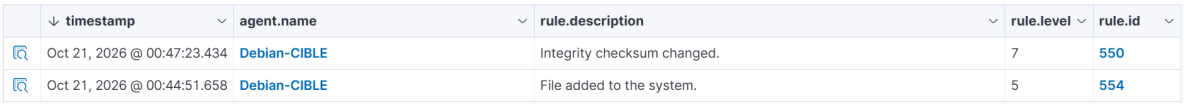
Détection des modifications par Wazuh

Comme le montre la figure 2.8, Wazuh a généré deux alertes sur l'agent 002 :

— **Règle 554, niveau 5 :** détection de la création d'un fichier à 00h44min51s.

— **Règle 550, niveau 7 :** détection d'une modification de l'empreinte d'intégrité à 00h47min23s.

La seconde alerte indique que le contenu d'un fichier surveillé a changé. Dans ce scénario, cette modification correspond à la mise à jour volontaire de la tâche cron.



	timestamp	agent.name	rule.description	rule.level	rule.id
	Oct 21, 2026 @ 00:47:23.434	Debian-CIBLE	Integrity checksum changed.	7	550
	Oct 21, 2026 @ 00:44:51.658	Debian-CIBLE	File added to the system.	5	554

Figure 2.8 – Détection de la création et de la modification d'un fichier par le module FIM de Wazuh

L'utilisation d'une tâche cron pour exécuter régulièrement un script peut correspondre à la technique MITRE ATT&CK `T1053.003` (*Cron*), notamment lorsqu'elle est utilisée pour maintenir un accès persistant à un système.

Toutefois, les alertes FIM signalent ici des modifications de fichiers et ne permettent pas, à elles seules, de confirmer une compromission.

Interprétation et investigation par l'agent IA

L'agent IA a ensuite été sollicité pour examiner les alertes de `Debian-CIBLE` entre 00h47 et 00h48.

Comme le montre la figure 2.9, Qwen3 a correctement identifié l'événement de niveau 7 enregistré à 00h47min23s et l'a interprété comme une possible altération de fichiers système.

L'investigation s'appuie sur trois appels aux outils : `search_alerts`, `get_agent_timeline` et `get_event_sequence`. Ces fonctions permettent de rechercher les alertes et de demander des informations sur leur contexte chronologique.

Le modèle a également mentionné des échecs d'authentification antérieurs et des cycles de démarrage et d'arrêt de l'agent Wazuh. Il a alors formulé l'hypothèse d'une tentative d'altération malveillante, éventuellement liée à une activité non autorisée.

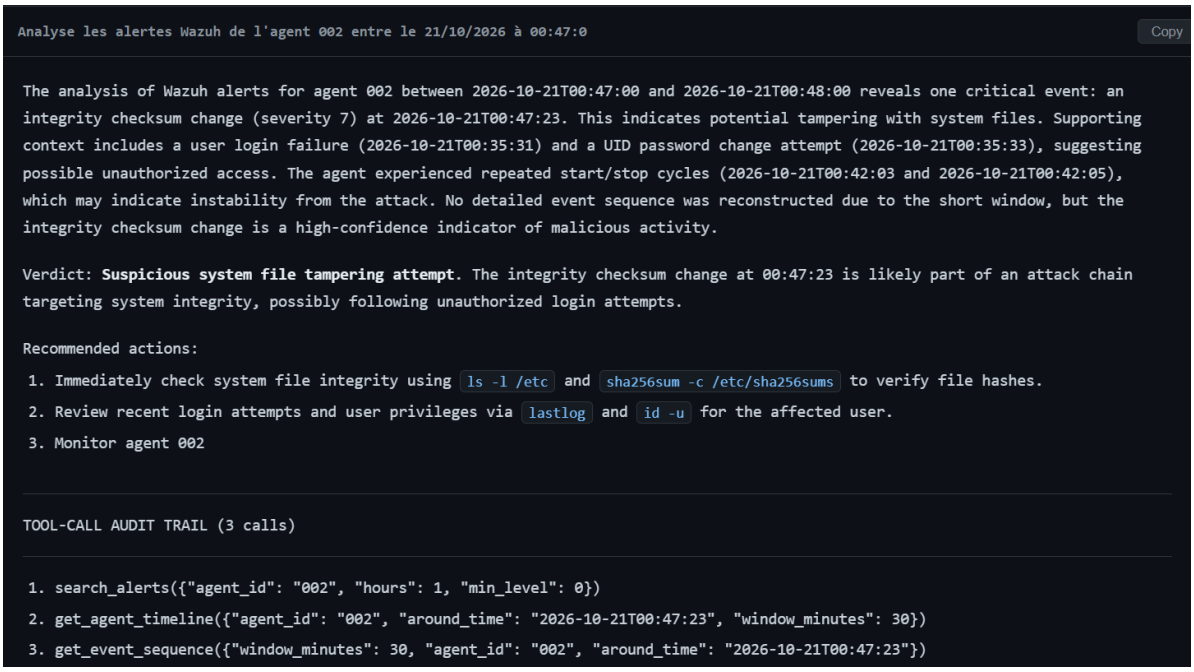


Figure 2.9 – Investigation de l'alerte FIM par Qwen3 et historique des appels aux outils

Sur le plan technique, Qwen3 a recommandé de vérifier l'intégrité des fichiers, d'examiner les connexions récentes et de contrôler les privilèges des utilisateurs. Ces propositions constituent des pistes d'investigation pertinentes pour un analyste SOC.

Cependant, l'analyse présente plusieurs limites. Le modèle n'a pas identifié précisément le fichier cron modifié et n'a pas établi de relation vérifiable entre cette modification et les échecs d'authentification précédents.

Ce scénario met en évidence deux aspects complémentaires de l'IA : sa capacité à **repérer un événement de sécurité et à proposer des vérifications techniques**, mais également son risque de **surinterprétation** lorsqu'elle ne dispose pas de suffisamment de contexte.

La validation humaine reste donc essentielle pour distinguer une modification légitime d'une véritable tentative de persistance.

2.5 Bilan des expérimentations et perspectives d'amélioration

Les expérimentations réalisées montrent l'intérêt d'associer Wazuh à un agent d'intelligence artificielle pour faciliter l'analyse des événements de sécurité.

Dans le scénario de force brute SSH rapide, Qwen3 a correctement interprété l'alerte générée par Wazuh et proposé des recommandations techniques, notamment le renforcement de l'authentification et le filtrage des connexions suspectes. Le mécanisme de *tool calling* permet également à l'agent IA d'interroger les événements et de conduire

une investigation à partir des données collectées.

Le résultat le plus intéressant concerne les tentatives SSH espacées dans le temps. Wazuh a enregistré les échecs d'authentification sans générer d'alerte spécifique de force brute pour cette séquence. Qwen3 a néanmoins reconnu un comportement compatible avec une attaque. Cette capacité peut contribuer à réduire les faux négatifs en attirant l'attention de l'analyste sur des comportements suspects qui ne déclenchent pas de règle de corrélation.

Le scénario de modification d'une tâche cron a également montré que l'IA pouvait identifier un changement d'intégrité et proposer des vérifications complémentaires. Cependant, Qwen3 a envisagé une activité malveillante alors que la modification était volontaire dans notre laboratoire. Cette interprétation reste compréhensible dans un contexte de cybersécurité : une modification inattendue d'un fichier sensible peut effectivement justifier une investigation. Le modèle a toutefois établi des liens avec d'autres événements sans disposer de preuves suffisantes pour confirmer une chaîne d'attaque.

Ces observations soulignent l'importance du contexte fourni à l'IA. La pertinence de ses conclusions dépend notamment des événements accessibles, de la précision des filtres temporels et des informations transmises par les outils d'investigation. Une recherche trop restrictive peut également exclure des alertes pertinentes.

Sur le plan technique, l'exécution locale de Qwen3 4B sur CPU permet de conserver les analyses dans l'environnement du laboratoire, mais peut entraîner des temps de réponse importants, particulièrement lorsque plusieurs appels aux outils sont nécessaires.

Pour améliorer le prototype, plusieurs évolutions pourraient être envisagées : l'ajout de règles Wazuh adaptées aux attaques à faible fréquence, l'amélioration du filtrage des événements par période et par agent, ainsi que l'intégration des preuves associées aux conclusions de Qwen3.

Enfin, l'agent IA doit être considéré comme un outil d'assistance à l'analyste SOC. Il facilite l'identification des comportements suspects, la compréhension des alertes et la proposition de mesures de sécurité. Les décisions de remédiation doivent cependant rester soumises à une validation humaine, notamment lorsque les conclusions reposent sur des hypothèses.

Ainsi, les résultats obtenus mettent en évidence la complémentarité entre la détection par règles de Wazuh et les capacités d'interprétation de Qwen3.

Conclusion

Les expérimentations montrent que l'apport de l'IA dépend du scénario et de la qualité des données. Lorsqu'une attaque est déjà reconnue par Wazuh, le modèle peut aider à expliquer l'alerte. Dans le cas SSH *low-and-slow*, il a reconnu un comportement suspect à partir d'échecs non corrélés en force brute par les règles observées. Cette valeur

ajoutée doit toutefois être nuancée par les erreurs factuelles et les recherches incomplètes relevées durant les essais.

Le scénario de modification d'une tâche cron a également montré la capacité de Qwen3 à interpréter une alerte d'intégrité et à proposer des vérifications techniques, malgré certaines hypothèses non confirmées. Ces résultats soulignent l'intérêt d'une approche complémentaire associant la détection par Wazuh, l'investigation assistée par l'IA et la validation humaine.

Conclusion générale

Ce projet a permis de mettre en place un laboratoire de supervision de sécurité associant un SIEM open source, Wazuh, à un agent d'intelligence artificielle local utilisant Qwen3. L'intégration repose sur la consultation des événements remontés par les agents et sur leur interprétation par le modèle, sans remplacer les mécanismes de collecte et de détection du SIEM.

Le résultat le plus significatif concerne les tentatives SSH espacées. Les échecs d'authentification ont été collectés par Wazuh, sans qu'une alerte de force brute corrélée ait été observée pour ce scénario. Qwen3 4B a néanmoins identifié un schéma compatible avec une tentative de force brute à faible fréquence. Cette observation montre l'intérêt potentiel de l'IA pour relier des signaux élémentaires et assister l'investigation. Elle ne démontre pas, à elle seule, une réduction générale des faux négatifs.

L'évaluation révèle également des limites : chronologie parfois incorrecte, qualifications MITRE non étayées, dépendance aux champs transmis par les outils de recherche et temps de réponse important sur CPU. L'IA doit donc être considérée comme un **outil d'aide à l'analyste**, et non comme un moteur de détection autonome dont les conclusions seraient systématiquement fiables.

Aucune règle personnalisée de corrélation SSH n'a été ajoutée durant l'étude : ce choix expérimental permettait d'observer la capacité du modèle à interpréter les alertes existantes. Dans un environnement de production, la démarche serait différente. Des règles Wazuh adaptées aux comportements à faible fréquence, une meilleure visibilité sur les événements, des contrôles de qualité des réponses et une validation humaine seraient nécessaires. La complémentarité entre détection déterministe, analyse assistée par IA et expertise humaine constitue ainsi la principale perspective de ce travail.

Bibliographie

[1] Wazuh, *Documentation officielle*, https://documentation.wazuh.com/current/.

[2] Ollama, *Documentation*, https://docs.ollama.com/.

[3] MITRE ATT&CK, *Enterprise Matrix*, https://attack.mitre.org/.

[4] Qwen, *Documentation des modèles*, https://qwenlm.github.io/.

[5] octopus237, *Agentic-AI, dépôt du prototype étudié*, https://github.com/octopus237/Agentic-AI.